

Library Management System Documentation

Member Table:

```
CREATE TABLE Member (  
    member_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    phone VARCHAR(15) UNIQUE,  
    membership_date DATE  
);
```

1.1 Author Table:

```
CREATE TABLE Author (  
    author_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100)  
);
```

1.2 Book Table:

```
CREATE TABLE Book (  
    book_id INT PRIMARY KEY AUTO_INCREMENT,  
    title VARCHAR(150),  
    author_id INT,  
    category VARCHAR(50),  
    price DECIMAL(10,2),  
    availability BOOLEAN,  
    FOREIGN KEY (author_id) REFERENCES Author(author_id)  
);
```

1.3 Issue Table:

```
CREATE TABLE Issue (  
    issue_id INT PRIMARY KEY AUTO_INCREMENT,  
    member_id INT,
```

```
book_id INT,  
issue_date DATE,  
return_date DATE,  
FOREIGN KEY (member_id) REFERENCES Member(member_id),  
FOREIGN KEY (book_id) REFERENCES Book(book_id)  
);
```

1.4 Fine Table:

```
CREATE TABLE Fine (  
fine_id INT PRIMARY KEY AUTO_INCREMENT,  
member_id INT,  
amount DECIMAL(10,2),  
paid_status BOOLEAN,  
FOREIGN KEY (member_id) REFERENCES Member(member_id)  
);
```

1.5 Staff Table:

```
CREATE TABLE Staff (  
staff_id INT PRIMARY KEY AUTO_INCREMENT,  
name VARCHAR(100),  
role VARCHAR(50)  
);
```

2 Insert Sample Data (10 Records Each)

2.1 Member:

```
INSERT INTO Member (name, phone, membership_date) VALUES  
( 'Kishan', '9876543210', '2024-01-10'),  
( 'Ravi', '9123456780', '2024-02-15'),  
( 'Sneha', '9988776655', '2024-03-05'),  
( 'Arjun', '9112233445', '2024-04-20'),  
( 'Meera', '9001122334', '2024-05-01'),  
( 'Rahul', '9887766554', '2024-05-10'),
```

('Priya', '9776655443', '2024-06-01'),
('Vikram', '9665544332', '2024-06-15'),
('Anita', '9554433221', '2024-07-01'),
('Suresh', '9443322110', '2024-07-10');

2.2 Author:

INSERT INTO Author (name) VALUES

('Chetan Bhagat'),
('J.K. Rowling'),
('George Orwell'),
('R.K. Narayan'),
('Jane Austen'),
('Mark Twain'),
('Agatha Christie'),
('Dan Brown'),
('Leo Tolstoy'),
('Paulo Coelho');

2.3 Book:

INSERT INTO Book (title, author_id, category, price, availability) VALUES

('Five Point Someone', 1, 'Fiction', 350, TRUE),
('Harry Potter', 2, 'Fantasy', 500, TRUE),
('1984', 3, 'Dystopian', 450, TRUE),
('Malgudi Days', 4, 'Short Stories', 300, TRUE),
('Pride and Prejudice', 5, 'Romance', 400, TRUE),
('Adventures of Tom Sawyer', 6, 'Adventure', 250, TRUE),
('Murder on the Orient Express', 7, 'Mystery', 350, TRUE),
('The Da Vinci Code', 8, 'Thriller', 600, TRUE),
('War and Peace', 9, 'Historical', 700, TRUE),
('The Alchemist', 10, 'Philosophy', 450, TRUE);

2.4 Issue:

```
INSERT INTO Issue (member_id, book_id, issue_date, return_date) VALUES
(1, 2, '2024-06-01', '2024-06-15'),
(2, 3, '2024-06-05', '2024-06-20'),
(3, 1, '2024-06-10', '2024-06-25'),
(4, 4, '2024-06-12', '2024-06-27'),
(5, 5, '2024-06-15', '2024-06-30'),
(6, 6, '2024-06-18', '2024-07-02'),
(7, 7, '2024-06-20', '2024-07-05'),
(8, 8, '2024-06-22', '2024-07-07'),
(9, 9, '2024-06-25', '2024-07-10'),
(10, 10, '2024-06-28', '2024-07-12');
```

2.5 Fine:

```
INSERT INTO Fine (member_id, amount, paid_status) VALUES
(1, 50, TRUE),
(2, 100, FALSE),
(3, 75, TRUE),
(4, 30, TRUE),
(5, 60, FALSE),
(6, 90, TRUE),
(7, 40, TRUE),
(8, 120, FALSE),
(9, 80, TRUE),
(10, 20, TRUE);
```

2.6 Staff:

```
INSERT INTO Staff (name, role) VALUES
('Ramesh', 'Librarian'),
('Geeta', 'Assistant'),
('Kiran', 'Manager'),
('Shweta', 'Clerk');
```

```
('Ajay', 'Technician'),  
('Sunita', 'Receptionist'),  
('Deepak', 'Security'),  
('Anjali', 'Cataloger'),  
('Manoj', 'Cleaner'),  
('Pooja', 'Accountant');
```

-- 1. Retrieve all books:

```
SELECT * FROM Book;
```

-- 2. Display books written by a specific author:

```
SELECT title FROM Book  
JOIN Author ON Book.author_id = Author.author_id  
WHERE Author.name = 'J.K. Rowling';
```

-- 3. Display issue and member details (INNER JOIN):

```
SELECT Issue.issue_id, Member.name, Issue.issue_date, Issue.return_date  
FROM Issue  
INNER JOIN Member ON Issue.member_id = Member.member_id;
```

-- 4. Display issue, member, and book details (3-table JOIN):

```
SELECT Issue.issue_id, Member.name, Book.title, Issue.issue_date, Issue.return_date  
FROM Issue  
INNER JOIN Member ON Issue.member_id = Member.member_id  
INNER JOIN Book ON Issue.book_id = Book.book_id;
```

-- 5. Count number of books in each category:

```
SELECT category, COUNT(*) AS total_books
```

FROM Book

GROUP BY category;

-- 6. Display categories having more than 50 books:

SELECT category, COUNT(*) AS total_books

FROM Book

GROUP BY category

HAVING COUNT(*) > 50;

-- 7. Retrieve books whose price is greater than the average book price

SELECT title, price

FROM Book

WHERE price > (SELECT AVG(price) FROM Book);